

Improving Training Data Quality for Automated Test Case Generation Using Machine Learning and Its Evaluation

Taiga Ishinari
Graduate Schools of Engineering
Nihon University
Koriyama, Fukushima, Japan
ceta25002@g.nihon-u.ac.jp

Kiyoshi Ueda*
College of Engineering
Nihon University
Koriyama, Fukushima, Japan
ueda.kiyoshi@nihon-u.ac.jp

Yuki Shimizu
Graduate Schools of Engineering
Nihon University
Koriyama, Fukushima, Japan
ceyu24010@g.nihon-u.ac.jp

Abstract

We aim to enhance the accuracy of automatic structuring of requirement specifications and automated test case generation by leveraging machine learning techniques. Training with large datasets often faces challenges such as overfitting, extended training time, and high computational cost. Thus, improving the quality of training data is a critical issue. In this study, we propose a training data generation method that can achieve accuracy comparable to that obtained from large-scale datasets, even when only a limited amount of training data is available. Our approach incorporates structured requirement specification documents, similar to test data requirement specifications, into the training set, and applies four different vectorization procedures. The effectiveness of the proposed method is validated through automated test case generation experiments. We quantitatively evaluate the quality of training data produced by the four procedures using two least-squares-based approximations: quadratic and logarithmic. The results demonstrate that selecting requirement specifications for training data with Latent Dirichlet Allocation (LDA) yields the highest-quality training data. These findings confirm the effectiveness of the proposed method for improving automated test case generation.

CCS Concepts

• Software and its engineering → Empirical software validation.

Keywords

automated test case generation, CRF, training data quality

ACM Reference Format:

Taiga Ishinari, Kiyoshi Ueda, and Yuki Shimizu. 2025. Improving Training Data Quality for Automated Test Case Generation Using Machine Learning and Its Evaluation. In *2025 The 8th International Conference on Computational Intelligence and Intelligent Systems (CIIS '2025)*, November 21–23, 2025, Okayama, Japan. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3787256.3787263>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CIIS '2025, Okayama, Japan*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1933-2/25/11
<https://doi.org/10.1145/3787256.3787263>

1 Introduction

As the scale and complexity of high-quality communication system software continue to increase, prolonged development periods and rising costs have become significant issues. For such systems, the waterfall model (particularly the V-model) development process shown in Figure 1 is commonly adopted. The processes in the lower layers of the V-model are highly automated.

For unit testing, there is some research on automated testing and automatic test case generation, and most developments commonly use many supporting tools [5]. Almost all tools analyze the source code and then generate test cases. However, there are no tools for system testing in communication node software development. Researchers have started to use analysis design such as Unified Modeling Language (UML) for test case generation [7]. UML uses formal language to describe specification documents by using some diagrams. However, for communication node software development, stakeholders often use their natural language rather than UML to exchange their idea, business processes, business rules, and other specifications and to describe the specifications in documents.

Automating the generation of system test cases from natural language requirement specifications in the higher-level processes remains a difficult and unsolved problem. Masuda et al. [11, 12] proposed a specification document analysis technique that can put each branch element into a decision table [6] and transform the tables into test cases. They developed an algorithm for the analysis and applied it to specification documents. Because specification documents written in natural language often lack many necessary words and have grammatically complex sentences, we introduce machine learning and artificial intelligence techniques for analysis.

To address this challenge, we previously proposed a method for automatically generating system test cases from requirement specifications by applying machine learning techniques [8], specifically Conditional Random Fields (CRF) [9].

As shown in Figure 2, our method first structures the requirement specification by tagging it (process from top to middle section), then extracts the input/output information necessary for test case generation (process from middle to bottom section). Currently, system development experts extract the input/output information sections from requirement specifications corresponding to system test cases. Our approach automates this process using machine learning.

This study proposes a method to improve the quality of training data in order to enhance the efficiency of machine learning. In addition to the monotonically increasing trend where accuracy improves as the number of requirement specifications included in the training data increases, it is desirable to obtain a trend where

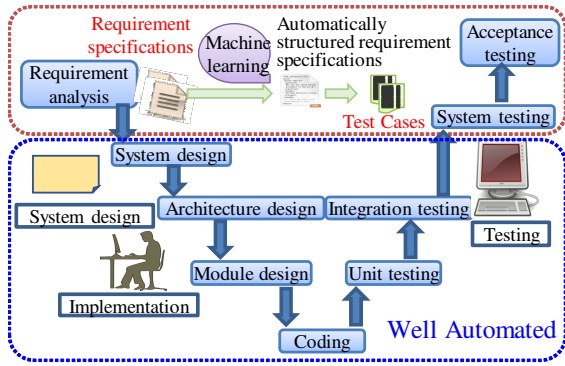


Figure 1: Automated System Test Case Generation in V-model.

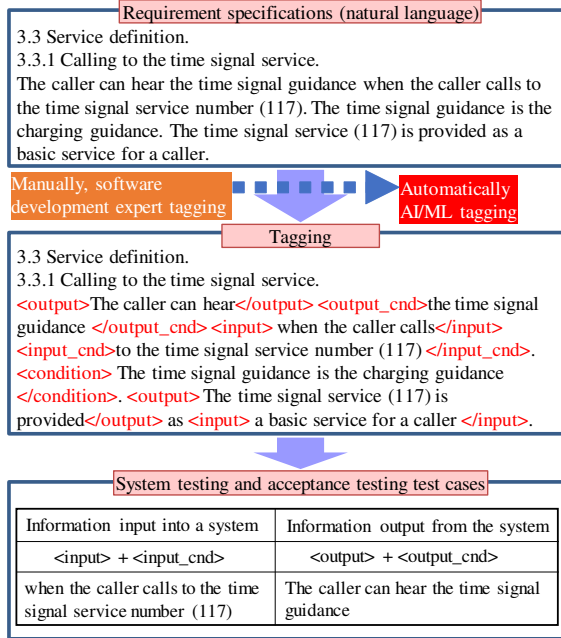


Figure 2: Structuring Requirements Specification and Extract Test Cases.

accuracy increases at a higher slope than linear growth even from a small number of requirement specifications. Experiments using actual requirement specifications from various IT systems are conducted to evaluate the proposed method.

2 Previous research

2.1 Automated Test Case Generation Method

Figure 3 shows the process flow of the method we previously proposed for automatically generating system test cases from requirements specifications.

Prior to training, system development experts manually tag and structure input/output information from multiple requirement specifications for machine learning training data. Figure 4 shows the

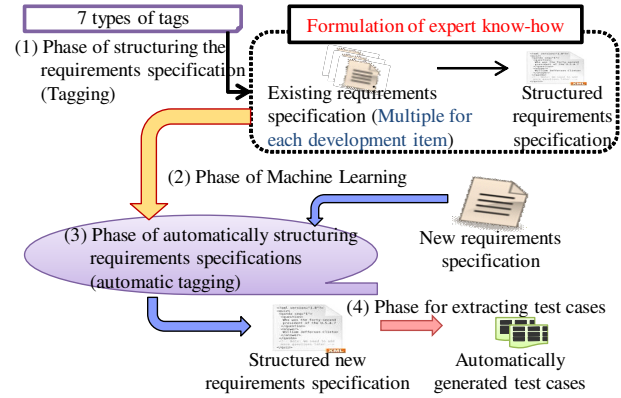


Figure 3: Flow of Automated Test Case Generation Method.

relationship between each tag type. The basic tag types are <input>, which indicates the trigger for system operation and input information, and <output>, which indicates execution results and output information. This method assigns an <input_cnd>tag when there is detailed information on the input, and an <output_cnd>tag when there is detailed information on the output. In the process, the following tags may be added: <condition>tag for system conditions, <agent>tag for system entities, and <check_point>tag for system check points.

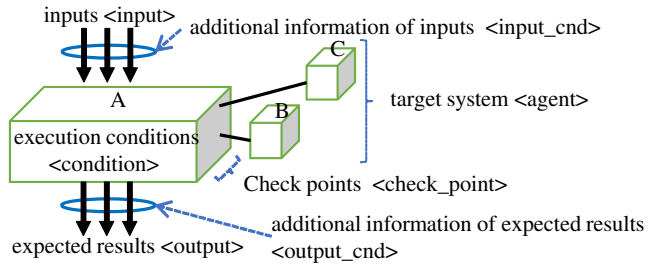


Figure 4: Document Structuring Tag Types.

Multiple structured requirement specifications are morpheme-analyzed using MeCab [10]. In the “Phase of machine learning”, this data is used to train a CRF model to learn relationships between tokenized words and their associated labels. Figure 5 shows an example of data processing according to the flow of the automatic test cases generation method. For example, the part enclosed in the <input_cnd>tag “AC address” is broken down into the morphemes “AC” and “address” by morphological analysis, and B-IC is assigned to “AC” and I-IC is assigned to “address”. The training data with tags assigned to each morpheme is then used to train the machine learning tool CRF++ [9]. At this time, a model file is generated that becomes the learning model for machine learning.

In the “Phase of automatically structuring requirement specifications”, new requirement specifications undergo morphological analysis via MeCab, and the trained CRF assigns appropriate tag labels to each word (i.e., sequence labeling) [8].

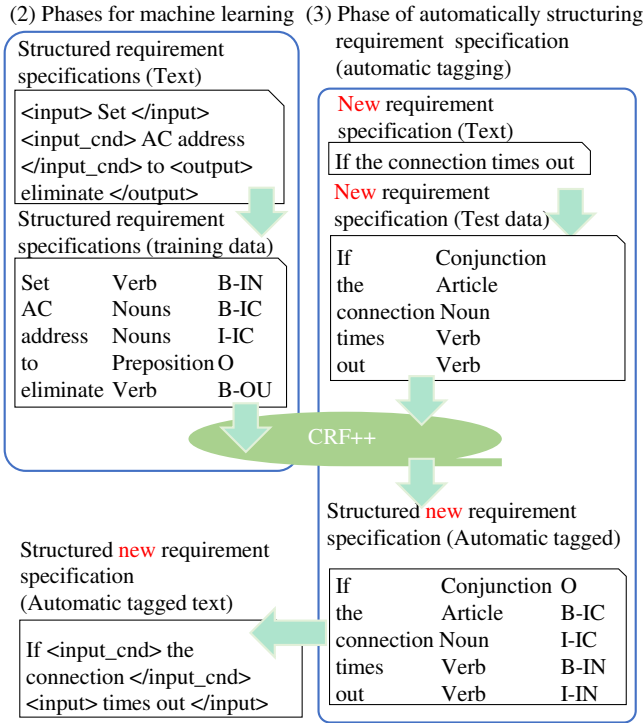


Figure 5: Automatically Structuring Data Processing.

2.2 Research Task

We aim to improve the accuracy of automatic structuring of requirement specifications and automatic generation of test cases. Generally, accuracy increases with more training data. Therefore, in addition to manually creating training data, we employ data augmentation techniques to generate large volumes of training data. However, when training on large datasets, efforts are needed to mitigate issues such as overfitting, training time, and computational cost. For automated test case generation, we also trained the CRF model using all existing structured requirement specification documents as training data. Generally, accuracy improved as the amount of training data increased. However, in our previous research, we observed cases where accuracy gradually declined after reaching a certain level despite further increases in training data [4, 19].

Therefore, it is necessary to establish a method for improving the quality of training data. Specifically, we aim to establish a training data generation method that achieves accuracy equivalent to that obtained when using all existing structured requirement specification documents as training data, even with a limited number of requirement specification documents as training data.

If the training data significantly differs from the new requirements specification (test data), it is likely that the desired input/output information words in the test data cannot be tagged, resulting in unrelated words being tagged instead. We consider it crucial to improve the quality of training data by excluding requirements specifications that lower the accuracy rate. In this study,

we hypothesized that accuracy would improve by including structured requirement specification documents similar to new test data requirement specifications in the training data.

3 Related Work

Related research has explored methods for constructing high-accuracy models even with limited training data. Safonova [16] provides an overview of ten deep learning techniques designed to address the small data problem, including transfer learning, self-supervised learning, and few-shot learning, highlighting how these methods mitigate overfitting and improve generalization. Zhang et al. [21] propose Deep Adversarial Data Augmentation (DADA), a GAN-based approach that generates class-conditional samples to improve classification performance in extremely low-data regimes, demonstrating superior generalization compared to traditional augmentation methods. These are methods for general text, and we need to consider their application to the labeling problem of extracting test cases from requirement specifications.

Additionally, the following researches have been reported regarding test case generation from natural language specifications. Mustafa et al. [15] analyze automated test case generation methods from requirements, covering approaches using UML, use cases, and natural language, and identifies challenges such as handling ambiguities in natural language requirements. Wang et al. [20] present an NLP-based approach for automatically generating acceptance test cases from natural language use case specifications, achieving a 95% success rate in mapping specification steps to test steps and uncovering previously overlooked scenarios. Fischbach et al. [2] propose an approach for automatically generating test models from semi-structured requirements by detecting structured fragments in natural language and converting them into Cause-Effect Graphs, reducing manual effort by up to 86%. Medeshetty et al. [14] apply NLP techniques, including named entity recognition (NER) and rule-based methods, to generate test cases for high-performance ECU systems. Results show rule-based methods perform better on simple requirements, while ML-based NER achieves higher accuracy on complex ones. The requirement specifications we target are not logically written natural language by engineers, but rather written by system ordering business personnel. Consequently, they frequently omit words and phrases, making it difficult for sentence structure analysis methods to automatically extract test cases that system development experts extract based on their experience-based know-how. Furthermore, with diverse IT systems, accuracy could decline if training data was not selected to match each system's characteristics. We need to establish a method that addresses this issue as well.

Saleem [17] investigates the use of generative language models such as ChatGPT and Gemini for requirement engineering tasks, including classification, named entity recognition, and question answering, and demonstrates that prompt design significantly affects model performance. While approaches utilizing LLMs are promising, given that LLMs are rapidly evolving technologies with concerns regarding information security and stability of accuracy, we concluded that establishing methods that do not rely on LLMs is also necessary.

4 Proposed Method

4.1 Selecting Requirement Specifications for Training Data by Vectorization

We have investigated a method for selecting multiple structured requirement specifications similar to the new test data requirement specification and incorporating them into the training data.

Several evaluation algorithms exist for measuring similarity between documents written in natural language. The fundamental approach involves converting text into vectors using methods like Bag-of-Words, Doc2Vec, or Transformers (BERT, RoBERTa, etc.), then measuring similarity based on the cosine angle between two vectors. Another approach utilizes high-dimensional vectors (embeds) provided by LLMs like OpenAI Embeddings, InstructorXL, or E5. For structured text like specifications, reports, or contracts, methods such as segmenting by section and calculating weighted averages of partial similarity, or local comparisons using Semantic Chunking + Embedding, are effective. However, these are not suitable for complex natural language, which often involves word omissions. This paper employs the fundamental method. We will explore similarity evaluation methods using LLMs in the future.

Each requirement specification is converted into a vector, and the similarity between requirement specifications is quantified using the cosine similarity between requirement specification vectors. Cosine similarity represents document similarity with values between 0 and 1, where higher similarity corresponds to larger values. The cosine similarity between the requirement specification vector q and d is obtained by equation (1).

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} \quad (1)$$

The Table 1 shows an example of a two-dimensional matrix representing the cosine similarity between each requirement specification document. Each row and column is each requirement specification document. The value located at the intersection of each requirement specification is the cosine similarity calculated by equation (1) using q , the vector representation of the requirement specification content in that row, and d , the vector representation of the requirement specification content in that column.

Multiple structured requirement specifications with high cosine similarity to the new requirement specification used for test data were selected and included in the training data.

If there is significant deviation in similarity between each requirement specification, it is meaningful to select similar requirement documents and include them in the training data. Conversely, if there is no deviation in similarity between each requirement specification, it can be said that selecting any requirement specification is meaningless. We selected new requirement specifications suitable for applying this method. We calculated the standard deviation of the cosine similarity values between the new requirement specifications vector and each structured specification vector. The proposed method is applied to new requirement documents belonging to the upper half of the standard deviation.

Table2 shows an example of calculating the standard deviation of cosine similarity between each new requirement specification vector and other requirement specification vectors. For example,

Table 1: Example of Cosine Similarity Values between each Requirement Specification Documents.

	A	B	C	D	E	F	G	H	I	J	K
A		0.16	0.30	0.28	0.36	0.24	0.35	0.26	0.34	0.45	0.56
B	0.16		0.14	0.13	0.12	0.21	0.29	0.08	0.12	0.16	0.21
C	0.30	0.14		0.23	0.21	0.23	0.36	0.21	0.20	0.23	0.30
D	0.28	0.13	0.23		0.20	0.28	0.26	0.14	0.23	0.26	0.32
E	0.36	0.12	0.21	0.20		0.18	0.24	0.36	0.20	0.35	0.35
F	0.24	0.21	0.23	0.28	0.18		0.42	0.14	0.19	0.29	0.42
G	0.35	0.29	0.36	0.26	0.24	0.42		0.18	0.23	0.33	0.42
H	0.26	0.08	0.21	0.14	0.36	0.14	0.18		0.17	0.27	0.24
I	0.34	0.12	0.20	0.23	0.20	0.19	0.23	0.17		0.32	0.37
J	0.45	0.16	0.23	0.26	0.35	0.29	0.33	0.27	0.32		0.64
K	0.56	0.21	0.30	0.32	0.35	0.42	0.42	0.24	0.37	0.64	

the first row calculates the standard deviation of cosine similarity between new requirement specification vector A and other requirement specification vectors B through T. If there are 20 new requirement specification documents from A to T, the proposed method is applied to the top half of the standard deviation, i.e., the 10 requirement specifications.

Table 2: Example of Calculating the Standard Deviation of Cosine Similarity Between Each New Test Data Requirement Specification and Other Requirement Specification.

	A	B	C	D	E	F	G	H	I	J	K	STD
A		0.16	0.30	0.28	0.36	0.24	0.35	0.26	0.34	0.45	0.56	0.10
B	0.16		0.14	0.13	0.12	0.21	0.29	0.08	0.12	0.16	0.21	0.06
C	0.30	0.14		0.23	0.21	0.23	0.36	0.21	0.20	0.23	0.30	0.06
D	0.28	0.13	0.23		0.20	0.28	0.26	0.14	0.23	0.26	0.32	0.08

4.2 Requirement Specification Document Vectorization Using Bag-of-Words

Each requirement specification was vectorized using a Bag-of-Words model based on the frequency of words (nouns in this case) that characterize each specification [19]. Each requirement specification document was morphologically analyzed, and TF-IDF was applied to assign weights to words representing features. Hereafter, this procedure is referred to as “BoW”.

4.3 Requirement Specification Document Vectorization Using Topic Analysis

We considered that there is a certain regularity in the context of words—the input and output information in test cases—and that these words and their contexts tend to differ depending on the development content specified in the requirements document. Therefore, it is necessary to examine a vectorization method for requirements documents that considers word order, inter-word distance, semantic representation of words, and descriptive tendencies.

We therefore hypothesized that topic analysis of sentences written in natural language could capture these tendencies. Topic models assume that a document is composed of multiple topics and statistically analyze the latent meaning within the document. Topic analysis is a method that derives the topic distribution (the probability of each topic appearing in a document) for each document based on word frequency. Documents can be classified by topic through topic analysis. For example, a document where similar terms like “politics”, “cabinet”, and “recession” frequently appear across multiple sentences would yield a “Politics and Economy” topic, while a document where terms like “baseball” and “soccer” frequently appear would yield a “Sports” topic. A representative examples of the topic model are Latent Semantic Analysis (LSA) [3] and Latent Dirichlet Allocation (LDA) [1].

LSA is a statistical model of word usage that permits comparisons of semantic similarity between pieces of textual information. LSA performs Singular Value Decomposition (SVD) on matrices of words and documents, enabling it to treat synonyms and semantically related words as belonging to the same topic. It extracts topics using a linear algebra-based method rather than a probabilistic model. Using LSA, each candidate requirement specification is converted into a document vector. Hereafter, this procedure is referred to as “LSA”.

Regarding LSA, the “LSA-noun-only” procedure is also performed for noun only. LSA-noun-only refers to the topic analysis performed using only noun when selecting requirement specifications to include in the training data, while the automatic structuring by CRF is identical to that performed for all parts of speech.

LDA is a three-level hierarchical Bayesian model, where each item in a collection is represented as a finite mixture model over an underlying topic set. Before calculating similarity between requirement specifications, remove unnecessary punctuation and noise. Next, tokenize the requirement specification documents using morphological analysis. Using LDA, each candidate requirement specification is converted into a document vector [18]. Hereafter, this procedure is referred to as “LDA”.

5 Evaluation

5.1 Evaluation Perspectives

We verify whether the proposed method BoW, LSA, LSA-noun-only, and LDA procedures, can achieve accuracy comparable to that obtained with all existing structured requirement specification documents as training data. Compare the accuracy of automated test case generation by sequentially increasing the number of requirement specifications included in the training data. Measure the accuracy of automated test case generation using training data containing the single requirement specification with the highest similarity to the test data. Next, measure the accuracy using training data containing the two documents with the highest similarity to the test data. Further, measure the accuracy by increasing the number of documents in the training data to three, four, and so on. If high accuracy is achieved with a small number of requirement specification documents, it indicates an improvement in the quality of the training data.

We evaluated their accuracy using 14 Japanese IT system requirement specifications listed in Table 3.

Table 3: List of Various IT System Requirement Specification Documents

Requirement specification document title	Words
Yokohama City Reception Management and Application Support System Development Specifications	426
Functional Overview of Information Sharing Infrastructure	1,400
Overview of Data Federation Infrastructure	293
Shimane Prefecture e-Procurement System Operation Manual	869
Firefighting Office Processing System Specifications	557
Membership Information Management System Requirement Definition Document	531
Kyoto City Hospital General Information System Specifications	623
Police General Investigation Information System Case Management Operations Program Specifications	783
Police General Investigation Information System Crime Statistics Operation Program Specifications	1,333
Table of Availability of New Library Information System Requirements Definition	2,306
Nutrition Management System Software Set Specifications	436
Akita General Hospital Core System Requirements Specifications	1,424
Matsue City Water Supply and Sewerage Bureau Water Supply Facility Management Mapping System Construction Work Request Specifications	249
Yaizu City Financial Accounting System Construction Work	3,009

To verify the practicality of automatic test case generation, we evaluate the accuracy per test case. Therefore, as shown in Figure 6, we integrated the test case input information B-IN and I-IN, B-IC and I-IC into IN, and output information B-OU and I-OU, B-OC and I-OC into OU for evaluation.

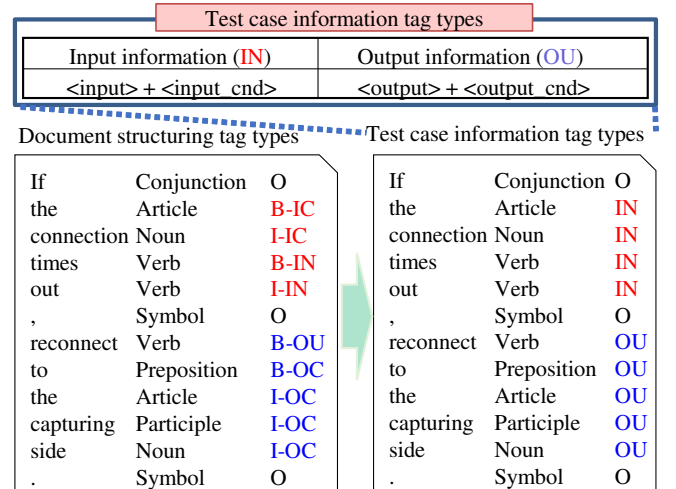


Figure 6: Conversion to Test Case Information Tag Types.

For each tag assigned to words in the specification documents, we obtain Precision, Recall and F-measure as shown in Figure 7. This paper observes the F-measure to evaluate the accuracy of automated test case generation.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_measure = \frac{2 * Precision * Recall}{Precision + Recall}$$

		Correct data	
		Tagged	Untagged
Automatic tagging result	Tagged	True Positive (TP)	False Positive (FP)
	Untagged	False Negative (FN)	True Negative (TN)

“Tagged” is a word tagged with IN or OU

“Untagged” is an untagged word (tagged with *Other* (O))

An example of classification

TP : Correct data = IN : Automatic tagging result = IN

FP : Correct data = O : Automatic tagging result = IN

FN : Correct data = OU : Automatic tagging result = O

TN : Correct data = O : Automatic tagging result = O

Figure 7: Precision, Recall and F-measure.

5.2 Evaluation Methods

We created software to analyze and vectorize the word frequency in requirement specifications for BoW, and calculated the cosine similarity between each requirement specification.

Using MATLAB [13], we created topic analysis software for LSA and LDA to vectorize the description of each requirement specification document and calculate the cosine similarity between each requirement specification document vector. When creating LSA and LDA models, it is necessary to set the number of topics. We considered that the greater the variation of the cosine similarity among the requirement document vectors, the more strongly they represent the characteristics of the requirement document. Therefore, we assume that the number of topics with the largest standard deviation of the total cosine similarity is optimal. We measured the standard deviation of the cosine similarity between vectors of the requirement documents for all of the number of topics. The standard deviation for the number of topics 5 was the highest, showing the characteristics of the documents. Therefore, the number of topics was set to 5 in this experiment. We vectorized each requirement specification into five topics and calculated the cosine similarity between each requirement specification vector.

The evaluation procedure for proposed method is as follows.

- (1) Vectorize all 14 requirement documents of various IT systems using each procedure of proposed method.
- (2) Calculate the cosine similarity between each requirement document vector and the other 13 requirement document vectors.
- (3) Calculate the standard deviation of the cosine similarity between each requirement document and each of the other requirement documents.
- (4) Use the top half (7 documents) of new requirement documents with the highest standard deviation of cosine similarity as test data. For each test data, sequentially input 1 to 13 structured requirement specification documents with high cosine similarity into the training data and perform automatic tagging using CRF++.

- (5) Calculate the F-measure of the automatic test case generation.

5.3 Evaluation Results

Figure 8 shows the F-measure for each procedure of proposed method on various IT system requirement specifications. The horizontal axis represents the number of structured requirement specifications included in the training data, while the vertical axis shows the average F-measure for the top 7 new requirement specifications.

With training data containing a small number of requirement specifications (1, 2, or 3), LDA showed the most rapid improvement in F-measure. This suggested LDA was effective for improving the quality of training data. Conversely, LSA-noun-only did not appear to achieve such qualitative improvement.

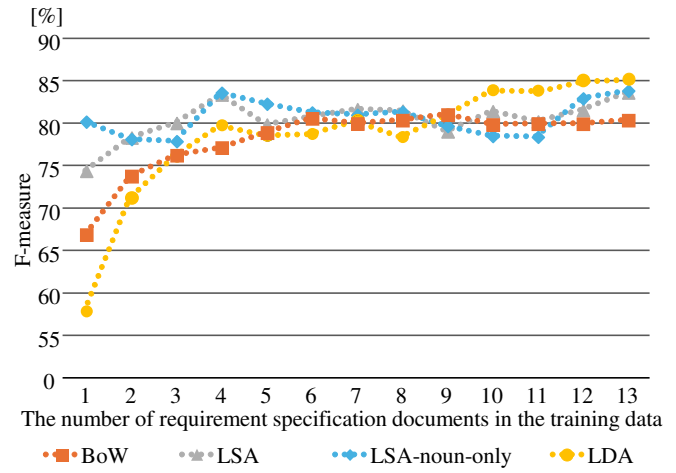


Figure 8: F-measure Relative to the Number of Structured Requirement Specifications to be Included in the Training Data.

The degree of usefulness of the proposed method is not expressed numerically. An evaluation method using a formula to quantitatively determine usefulness is necessary. This study observes the degree to which the quality of training data improves, thereby enhancing the efficiency of machine learning—the primary objective of this research. It quantitatively demonstrates the extent to which accuracy increases at a gradient higher than linear growth, even at stages with few requirements, in addition to the monotonic increase trend where accuracy improves as the number of requirements in the training data increases.

Using quadratic approximation and logarithmic approximation formulas by least squares method, we quantify improvements in training data quality and verify their usefulness. For each procedure in the proposed method, we compare the coefficient of x^2 in the quadratic approximation and the coefficient of $\ln$ in the logarithmic approximation. We verify whether accuracy improves rapidly with a limited number of requirement specification documents as training data. A larger negative value for the coefficient of x^2 in a quadratic function and a larger positive value for the coefficient of the $\ln$

serve as good indicator values for rapid F-measure improvement and the graph has significant curvature.

Figure 9 shows examples of quadratic approximation and logarithmic approximation for LDA and LSA noun-only procedures. Both metrics quantitatively indicate that LDA performs better than LSA-noun-only.

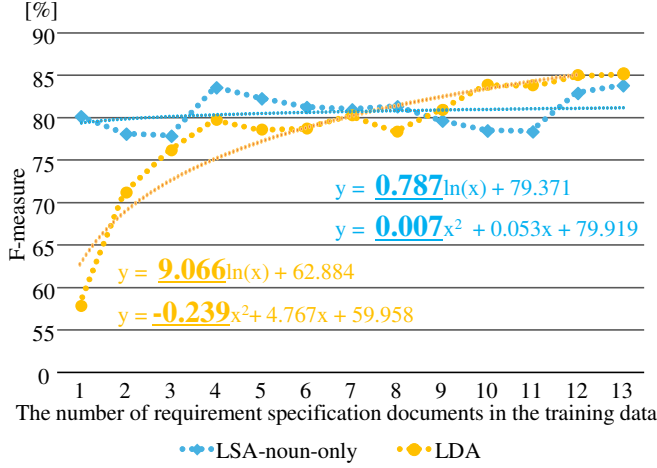


Figure 9: Quadratic Approximation and Logarithmic Approximation Formulas.

As shown in Figure 10, LDA exhibited the largest negative coefficient values, confirming the highest quality of training data.

Furthermore, Figure 8 shows that both LSA and LSA-noun-only outperform BoW in terms of F-measure. However, in the graph of Figure 10, the coefficients for LSA-noun-only are closest to zero. Even within LSA, it was confirmed that the values are closer to zero and the graph has less curvature compared to the BoW method.

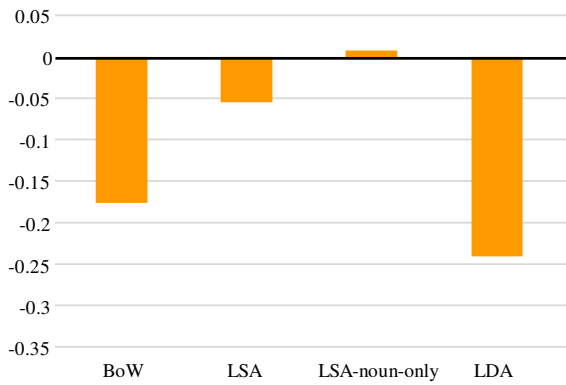


Figure 10: Coefficient of x^2 in the Quadratic Approximation.

In Figure 11, comparing the coefficients of $\ln$ in the logarithmic approximation for each procedure confirms the curvature of the graph. LDA has the largest $\ln$ coefficient, confirming it has the highest quality training data. Conversely, LSA-noun-only has the lowest

$\ln$ coefficient, consistent with the comparison of the coefficients of x^2 in the quadratic approximation.

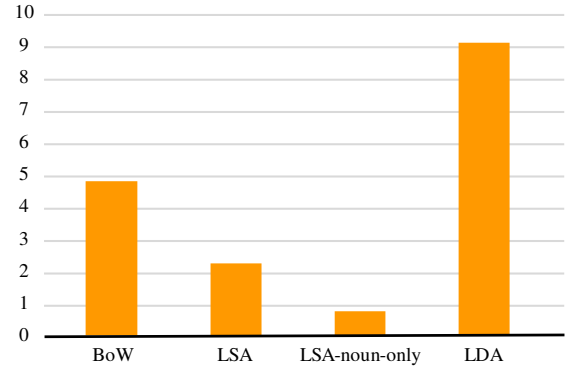


Figure 11: Coefficient of $\ln$ in the Logarithmic Approximation.

5.4 Consideration of Evaluation Results

In this study, we quantitatively compared the quality of training data using the proposed method's four procedures with two evaluation techniques: quadratic approximation and logarithmic approximation. Figures 10 and 11 confirm that LDA produced the highest quality training data. This is because LSA is a matrix-based topic extraction method that inherently handles only synonym topics. In contrast, LDA employs Bayesian estimation, enabling it to extract topics with similar meanings independently of existing documents. This flexibility of LDA is considered to have contributed to the improved accuracy. Furthermore, the LSA-noun-only method, which performs topic analysis using only nouns, yielded the worst results. A review of the target parts of speech for topic analysis is considered necessary.

Moreover, Figure 8 shows that LDA achieved higher maximum accuracy than BoW. The reason is that BoW does not consider the semantic representation of sentences, leading to an inability to handle semantically similar alternative words. Conversely, LDA can account for semantic relevance, which likely resulted in improved accuracy.

There were instances where LSA accuracy appeared lower than BoW. BoW directly utilizes word frequency and employs all words as features, minimizing information loss. In contrast, LSA has disadvantages: unlike LDA, it cannot handle polysemous words, and its use of SVD for dimensionality reduction compresses information, potentially causing loss of important details. This likely resulted in lower accuracy.

LDA achieved high F-measure exceeding 85%, confirming that the test case auto-generation method is viable for practical system software development.

It is desirable to consider the scale and diversity of the IT system requirements specifications used for evaluation. A larger scale is considered preferable, and we plan to increase the volume using training data augmentation techniques. Regarding diversity, we believe that preparing more diverse specifications will enable the

selection of specifications with higher similarity to the test data. We will further investigate and collect publicly available specifications to prepare more diverse requirements specifications.

6 Conclusion

In this study, we addressed the challenge of improving the accuracy of automatic structuring of requirement specifications and automated test case generation. Since training on large datasets often incurs problems such as overfitting, long training time, and high computational cost, we focused on enhancing the quality of training data to achieve high accuracy with a limited number of requirement specification documents as training data. We proposed four training data generation procedures - BoW, LSA, LSA-noun-only, and LDA - that incorporate structured requirement specification documents into the training set. Through quantitative evaluation using quadratic and logarithmic least-squares approximations, we confirmed that LDA produced the highest-quality training data, achieving an F-measure of over 85%. These results demonstrate the effectiveness of the proposed approach in supporting automated test case generation.

Future work includes comparing accuracy across different numbers of requirement specification documents and varying contents, as such factors may influence the performance of the proposed method.

Acknowledgments

We would like to thank Yuka Komatsu, a former member of our laboratory, for her great efforts.

References

- [1] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent dirichlet allocation. *J. Mach. Learn. Res.* 3, null (March 2003), 993–1022.
- [2] Jannik Fischbach, Maximilian Junker, Andreas Vogelsang, and Dietmar Freudenstein. 2019. Automated Generation of Test Models from Semi-Structured Requirements. (2019), 263–269. doi:10.1109/REW.2019.00053
- [3] Peter W. Foltz. 1996. Latent Semantic Analysis for text-based research. *Behavior Research Methods, Instruments, & Computers* 28, 2 (June 1996), 197–202.
- [4] Yuto Fujita and Kiyoshi Ueda. 2024. A Method for Selecting Training Data Using Doc2Vec for Automatic Test Cases Generation. In *2024 IEEE International Conference on Consumer Electronics*. 1–6. doi:10.1109/ICCE59016.2024.10444275
- [5] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) (PLDI '05). Association for Computing Machinery, New York, NY, USA, 213–223. doi:10.1145/1065010.1065036
- [6] ISO 29119-4 2021. Software and system engineering - software testing - part 4 Test techniques.
- [7] Pariksha Jain and Dinesh Soni. 2020. A Survey on Generation of Test Cases using UML Diagrams. In *2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE)*. 1–6. doi:10.1109/ic-ETITE47903.2020.395
- [8] Kazuhiro Kikuma, Takeshi Yamada, Koki Sato, and Kiyoshi Ueda. 2019. Preparation Method in Automated Test Case Generation using Machine Learning. In *Proceedings of the 10th International Symposium on Information and Communication Technology* (Hanoi, Ha Long Bay, Viet Nam) (SoICT '19). Association for Computing Machinery, New York, NY, USA, 393–398. doi:10.1145/3368926.3369679
- [9] Taku Kudo. 2013. CRF++: Yet Another CRF toolkit. Retrieved Sept. 4, 2025 from <http://taku910.github.io/crfpp/#source>
- [10] Taku Kudo. 2013. MeCab: Yet Another Part-of-Speech and Morphological Analyzer. Retrieved Sept. 4, 2025 from <http://taku910.github.io/mecab/>
- [11] Satoshi Masuda, Futoshi Iwama, Nobuhiro Hosokawa, Tohru Matsuodani, and Kazuhiko Tsuda. 2015. Semantic analysis technique of logics retrieval for software testing from specification documents. In *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 1–6. doi:10.1109/ICSTW.2015.7107430
- [12] Satoshi MASUDA, Tohru MATSUODANI, and Kazuhiko TSUDA. 2016. Detecting Logical Inconsistencies by Clustering Technique in Natural Language Requirements. *IEICE Transactions on Information and Systems* E99.D, 9 (2016), 2210–2218. doi:10.1587/transinf.2015KBP0005
- [13] MathWorks. 2025. MATLAB. <https://jp.mathworks.com/products/matlab.html>
- [14] Nikitha Medeshetty, Ahmad Nauman Ghazi, Sadi Alawadi, and Fahed Alkhabbas. 2025. From Requirements to Test Cases: An NLP-Based Approach for High-Performance ECU Test Case Automation. (2025). arXiv:2505.00547 [cs.SE] <https://arxiv.org/abs/2505.00547>
- [15] Ahmad Mustafa, Wan M. N. Wan-Kadir, Noraini Ibrahim, Muhammad Arif Shah, Muhammad Younas, Atif Khan, Mahdi Zareei, and Faisal Alanazi. 2021. Automated Test Case Generation from Requirements: A Systematic Literature Review. *Computers, Materials and Continua* 67, 2 (2021), 1819–1833. doi:10.32604/cmc.2021.014391
- [16] Anastasiia Safonova, Gohar Ghazaryan, Stefan Stiller, Magdalena Main-Knorn, Claas Nendel, and Masahiro Ryo. 2023. Ten deep learning techniques to address small data problems with remote sensing. *International Journal of Applied Earth Observation and Geoinformation* 125 (2023), 103569. doi:10.1016/j.jag.2023.103569
- [17] Summra Saleem, Muhammad Nabeel Asim, Ludger Van Elst, and Andreas Dengel. 2025. Generative language models potential for requirement engineering tasks. *Springer* (2025).
- [18] Kiyoshi Ueda, Riku Shikama, and Yuki Shimizu. 2024. Automatic Test Cases Generation with Selection of Training Data for Various System Specifications. In *2024 6th International Conference on Computer Communication and the Internet (ICCCI)*. 1–5. doi:10.1109/ICCCI62159.2024.10674503
- [19] Kiyoshi Ueda and Hikaru Tsukada. 2021. Accuracy Improvement by Training Data Selection in Automatic Test Cases Generation Method. In *2021 9th International Conference on Information and Education Technology (ICIET)*. 438–442. doi:10.1109/ICIET51873.2021.9419636
- [20] Chunhui Wang, Fabrizio Pastore, Arda Goknil, and Lionel C. Briand. 2022. Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach. *IEEE Transactions on Software Engineering* 48, 02 (Feb. 2022), 585–616. doi:10.1109/TSE.2020.2998503
- [21] Xiaofeng Zhang, Zhangyang Wang, Dong Liu, Qifeng Lin, and Qing Ling. 2021. Deep Adversarial Data Augmentation for Extremely Low Data Regimes. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 1 (2021), 15–28. doi:10.1109/TCSVT.2020.2967419